
HungerHeal

Interview questions and answers

150 questions with answers

12 sections

Event-driven microservices · Go · Kafka · Redis

HungerHeal — 150 interview questions, with answers

Prep material for talking about this project. Answers are **what is actually true of this codebase**, not generic textbook lines — an interviewer who pushes one level deeper should find the code matches.

How to use this: the answers are written out fully so you can check your understanding, but do not recite them. In the room, lead with the one-sentence version and let them pull the detail. Where an answer names a number or a file, know it — those are the places a follow-up lands.

The three things that carry the whole interview, if you remember nothing else:

1. **The problem was a human bottleneck.** An admin manually assigned every donation to a collection agent. The rebuild replaced that person with a multi-parameter scoring engine.
2. **Distance is 35% of the decision, not 100%.** That one line demonstrates the whole design, and it is *demonstrable* — a farther, better-equipped agent beats a closer, unequipped one.
3. **You can say why any decision was made.** Every candidate's full score breakdown is stored and shown, including the ones who lost.

Contents

#	Section
A	Opening and narrative (1–12)
B	Problem, product and scope (13–22)
C	Microservices fundamentals (23–38)
D	Kafka and event-driven design (39–56)
E	Redis (57–72)
F	The matching algorithm (73–92)
G	Go (93–102)
H	MongoDB and data modelling (103–112)
I	Cassandra and analytics (113–120)
J	Gateway, auth and security (121–132)
K	Frontend (133–138)
L	Testing, debugging and war stories (139–150)

Quick reference — the numbers to know

Services	9 (7 core + gateway + optional analytics)
Written in Go	2 — agent-location-service , assignment-engine
Containers	15
Unit tests	246
Smoke checks	333 across 13 suites
Scoring weights	distance 0.35 , category 0.25 , load 0.20 , rating 0.15
Max possible score	0.95 (the weights sum to 0.95, deliberately)
Agents offered per round	3 , in parallel
Response window	90s high urgency · 3 min medium · 5 min low
Radius ladder (high urgency)	5 → 12 → 25 → 50 km
Agent heartbeat TTL	120s , reported every 20s
Geocode cache TTL	30 days
Outbox sweeper	every 15s
Gateway rate limits	300/min general · 20/min failed auth
Go image size	30MB vs 286MB for Node

Kafka topics: donation.created , donation.assigned , donation.accepted , donation.rejected , donation.timeout , donation.unassigned

Statuses: PENDING_ASSIGNMENT → OFFERED → ACCEPTED → COLLECTED , plus UNASSIGNED , CANCELLED , EXPIRED

Roles: DONOR , AGENT , ADMIN (read-only)

Categories: COOKED_PREPARED , PERISHABLE_RAW , BAKERY , PACKAGED_NON_PERISHABLE , BEVERAGES

A — Opening and narrative (1–12)

1 Tell me about your project.

HungerHeal is a food-donation platform that connects people with surplus food to volunteers who collect it. I rebuilt it from a monolithic MERN application into nine event-driven microservices.

*The reason for the rebuild was one specific problem. In the original, an **administrator had to manually assign every donation to a collection agent** — a human reading each donation and deciding who should pick it up. That is a bottleneck that gets worst exactly where it matters most, because perishable food sits waiting for someone to look at a screen.*

So the defining change is that the human is gone from the assignment path entirely. A donation is posted, an event goes onto Kafka, a Go service runs a geospatial query in Redis for agents on shift nearby, scores them on four weighted factors, and offers the donation to the top three in parallel — typically inside a second or two. Whoever accepts first gets it.

*The part I am most pleased with is that the decision is **explainable**. Every candidate's full score breakdown is kept, so you can open any donation and see exactly why each agent won or lost.*

Then stop. That is 60 seconds. Let them choose the direction.

2 Why microservices? Wasn't a monolith fine?

Honest answer, in this order:

1. **The learning goal was explicit.** This was a rebuild to demonstrate distributed-systems skills, and saying otherwise would be dishonest.
2. **But the split does earn its place in one concrete way:** three services — tracking, notification, analytics — were added **without changing a single line of any existing service**. A new Kafka consumer group receives the whole stream. In the monolith each of those would have meant editing the donation flow.
3. **The cost is real and I would name it:** nine services, 15 containers, and a whole class of failure that did not exist before — partial failure, duplicate delivery, eventual consistency. For a system at this traffic level a monolith would genuinely be less work.

Saying point 3 out loud is what makes points 1 and 2 credible.

3 What was the hardest part?

The **offer lifecycle**, because it is where three hard problems meet:

- Three agents are offered the same donation simultaneously and exactly one may win — a distributed race.
- If nobody answers, a timeout must fire with no user request to hang it on.
- Kafka delivers at least once, so every one of those transitions can be replayed.

Getting one of those right is easy. Getting all three right together, and proving it with a test that waits out a real 90-second timeout, took the longest.

4 What are you proudest of?

That the system can **explain its decisions**. Replacing a human decision-maker with an algorithm is only an improvement if someone can still answer "why did this go to him?". The monitoring view shows every scored candidate with all four weighted terms — winners *and* losers — so a bad assignment can be diagnosed and the scoring changed, instead of being worked around one donation at a time.

5 What would you do differently?

Three things, in order of how much I mean them:

1. **Fewer services.** Notification and tracking could reasonably be one. I split them to practise the pattern, not because the domain demanded it.
2. **Learn the scoring weights instead of hand-picking them.** They are reasoned heuristics. I now collect the acceptance data that would let me fit them.
3. **Add a dead-letter topic.** Right now an unparseable message is logged and committed, because the alternative — blocking the partition forever — is worse. Neither is right; a DLQ is.

6 How long did it take, and how did you work?

Phase by phase, twelve phases, each with its own document written at the time — `docs/00-` through `docs/12-` . Each phase ended with the service working, unit tested, and driven end to end by a smoke script against the running stack before I moved on.

That discipline mattered most on the Go services: I wrote the entire scoring package and got its 35 tests passing **with Docker stopped**, because that package imports nothing but the standard library.

7 Did you use AI to build this?

Answer it straight, then move to what you can defend:

Yes, I used an AI assistant as a pair-programmer throughout, and I would work that way again. What I can tell you is that I understand every decision in it and I can defend the tradeoffs — which is what the rest of this conversation will show. Ask me why the weights sum to 0.95, or why there is no assign button on the monitoring view, and you will get the actual reasoning.

Never pretend otherwise, and never hide behind it. The defence is demonstrable depth, and the bug stories in section L are the strongest proof: you cannot narrate a debugging session you did not live through.

8 Walk me through what happens when a donor posts a donation.

1. Browser POSTs multipart form data to the gateway at `/api/donations` . The gateway verifies the JWT once and proxies to donation-service.
2. donation-service validates, saves to MongoDB with `eventPublished: false` , and returns **201 immediately**.
3. It then publishes `donation.created` to Kafka, keyed by `donationId`.
4. assignment-engine consumes it, deduplicates on `eventId` via Redis `SETNX` , runs `GEOSEARCH` on `agents:live` for agents within 5km.
5. It filters out anyone unavailable or who does not carry that category — hard filters, before scoring.

6. It scores the rest, takes the top three, writes an offer record and a deadline into Redis, and publishes `donation.assigned`.

7. tracking-service and notification-service both consume that event independently. The agents' dashboards poll and the offer appears with a live countdown.

Total, about a second and a half. No human anywhere.

9 What happens if two agents accept at the same moment?

Exactly one wins, decided by a Redis `SETNX` on `donation:<id>:claimed`. `SETNX` is atomic — it sets the key only if it does not exist — so whoever's command reaches Redis first gets `true` and everyone else gets `false`.

The losers receive **HTTP 409**, and the UI renders that as *"another agent accepted this one first"*. That framing is deliberate: losing a race you were invited into is a **normal outcome**, not an error. Three agents were asked on purpose.

10 Why not just assign to the single nearest agent?

Because nearest is not the same as best, and I can demonstrate it. In the smoke test the closest agent of all carries only packaged goods — sending them a hot meal produces a delivery nobody can serve.

Distance is 35% of the score. Category fit is 25%, current load 20%, rating 15%. A farther, properly equipped agent beats a closer, unequipped one:

```
equipped(0.809) beat barenear(0.735) -- 1.1km farther, and still won
```

Load matters for a subtler reason: without it, every donation stacks onto whoever happens to be nearest until they are overwhelmed while everyone else sits idle.

11 How do you know it works?

246 unit tests and 333 smoke checks across 13 suites, and the split is deliberate:

- **Unit tests need no infrastructure.** The scoring package and the status state machine import nothing but the standard library, so a failure means the *algorithm* is wrong — never that a broker was slow.
- **Smoke tests drive the real stack** — reading events back out of Kafka, asserting on raw Redis structures, and waiting out an actual 90-second timeout.

I can also name the bugs the tests caught, and one they did not. That is section L, and it is the most interesting part.

12 If I gave you a week, what would you add?

In priority order, with reasons:

1. **A dead-letter topic**, because the current behaviour on a malformed message is the lesser of two bad options.
2. **Real notification delivery.** The events already carry `priority: HIGH` on the ones that justify waking a phone; nothing delivers them. It is in-app only today.
3. **Travel time instead of straight-line distance.** More accurate, at the cost of a network call per candidate on the hot path — which is why I did not do it yet.

B — Problem, product and scope (13–22)

13 Who are the users?

Three roles, and the third is the interesting one:

- **DONOR** — restaurants, canteens, caterers, households with surplus. They post food and track it.
- **AGENT** — volunteers who collect. They go on shift, receive offers, accept, and mark collected.
- **ADMIN** — **read-only**. Can see every matching decision and change none of them.

14 Why is ADMIN read-only? Surely an admin should be able to intervene.

This is the single best question to get, because the answer is the thesis.

An override button would be used. The first time a donation looked slow, somebody would press it — and the manual assignment this entire rebuild removed would be back, now with a nicer dashboard around it.

The right response to a bad match is to open the score breakdown, see which of the four terms produced it, and change the scoring so every future donation benefits. Fixing one case by hand fixes exactly one case.

If pushed on genuine emergencies: that is a fair challenge, and the honest answer is that a real deployment would need a break-glass path — audited, rate-limited, and requiring a reason to be recorded. What it must not be is an ordinary button on the main screen.

15 What does a donation actually contain?

Title, optional description, category, quantity (amount + unit from `SERVINGS` / `KG` / `ITEMS` / `LITRES`), pickup address, coordinates, a best-before timestamp, and up to five photos.

The **category is not cosmetic** — it decides which agents are eligible at all and how long they get to respond.

16 Why does the donor set a best-before time?

Because it is the only thing that says how much time the system actually has. The UI defaults it to six hours out, which is realistic for cooked food and saves the donor typing a date in the common case. The donor dashboard surfaces a warning when a donation is still uncollected with under two hours left — a donation with two hours of life is a different situation from one posted a minute ago.

17 What happens if no agent is available?

It is **announced, not dropped**. The engine widens the search through the radius ladder, and if the last rung finds nobody it publishes `donation.unassigned` with a machine-readable reason:

Reason	Meaning	Retryable
NO_AGENTS_FOUND	Nobody on shift in range	Yes — agents may come online
NO_ELIGIBLE_AGENTS	Agents found, none carry this category	Yes
ALL_AGENTS_EXHAUSTED	Everyone in range already declined or ignored it	No
NOT_GEOCODED	No coordinates, so no geospatial query is possible	—

Distinguishing "nobody was online" from "everyone declined" is what tells you whether to recruit agents or revisit the scoring. The donor sees *"Needs attention"* with the reason in words.

18 Why let the donor place a pin rather than just typing an address?

A typed address geocodes to the *middle of a long road*. An agent sent to the wrong end of it wastes a trip on food that may not keep. So the picker offers three routes — type an address, click or drag the pin, or use device location — and whichever is used, exact coordinates go with the donation.

Sending coordinates also means donation-service does not have to geocode the address itself, which spends provider quota on a less precise answer.

19 Why does the UI show the geocoding confidence?

Because a vague address resolves to a plausible-looking but wrong point, and the donor is the only person who can catch it. Confidence ≥ 7 shows *"precise match"*; below that, *"rough match, please check the pin"*.

20 There's no partner-NGO directory. Why not?

Because there is no organisation model in the system, and I was not willing to fake one.

Donations are matched to individual collection agents, who take them on to the kitchens they serve. Receiving organisations are not registered on the platform, so there is no partner list to show and no count of meals delivered to them. A grid of plausible-looking charity names would have been the easiest thing on the site to add and completely untrue.

The page explains the real chain instead and says where the gap is. Registering receiving organisations, and tracking the drop-off as a fourth status, is the natural next step.

21 Why doesn't the homepage show total meals rescued?

Same reason. Every statistics endpoint is authenticated and per-user, so there is no public aggregate — and the system has served test data, so any number I printed would be invented. Real figures appear on the dashboard once you sign in.

22 What's deliberately out of scope?

- **Payments** — nothing is bought or sold.
- **Ratings submitted by users.** Agents have a rating that feeds the score, but nothing writes to it yet; new agents start at a neutral 3.5.
- **Cancel-by-donor.** CANCELLED exists in the state machine and is tested; nothing publishes it.

- **HTTPS.** A real deployment terminates TLS at the ingress in front of the gateway. Self-signed certs locally would demonstrate nothing and complicate every `curl` in the project.
-

C — Microservices fundamentals (23–38)

23 List the services and what each owns.

Service	Port	Language	Owns
api-gateway	4000	Node	Routing, edge JWT verification, rate limiting
auth-service	4001	Node	Users, credentials, agent capabilities (Mongo)
donation-service	4002	Node	Donations, photo uploads, the outbox (Mongo)
geocoding-service	4003	Node	Address ↔ coordinates, cache (Redis)
agent-location-service	4004	Go	Live positions, presence, load (Redis)
assignment-engine	4005	Go	Scoring, offers, timeouts
tracking-service	4006	Node	Donation status, timeline, monitoring reads (Mongo)
notification-service	4007	Node	In-app notifications (Mongo)
analytics-service	4008	Node	Historical event store (Cassandra)

24 How did you decide those boundaries?

By **what data each one owns**, not by layer. Every service has its own database, and no service reads another's tables. The test is: if this service were down, what exactly stops working? If the answer is "nothing specific", the boundary is wrong.

Two boundaries I would defend hardest:

- **geocoding-service is separate** because it is the only thing that talks to a paid third-party API. Isolating it means one cache, one rate limit, one place where quota is spent, and one provider abstraction to swap.
- **assignment-engine is separate** because it is the product's whole thesis. It has no database of its own — it reads Redis, consumes events, publishes events.

And one I would not defend: tracking and notification could be one service.

25 Database per service — why, and what does it cost?

Why: a shared database is a shared schema, and a shared schema means no service can change its storage without coordinating with every other. That is a monolith with extra network hops.

What it costs: no joins and no foreign keys across services. To show a donor their donation *with* the agent's name, the name has to be **copied onto the tracking record** when the assignment happens. That is denormalisation by necessity, and it is why `assignedAgentName` and `assignedAgentPhone` live on the tracking document.

26 How do services find each other?

Docker's built-in DNS on the compose network — `http://auth-service:4001` resolves by service name. That is all this needs. Consul or Kubernetes DNS is the answer at a scale this is not at, and adding it would be machinery without a problem.

27 What's your service-to-service communication style?

Events for everything that has happened; HTTP only for asking a question.

- Facts go on Kafka: `donation.created`, `donation.accepted` and so on. The publisher does not know or care who consumes them.
- The two synchronous calls that exist are genuine queries: donation-service asks geocoding-service to resolve an address, and agent-location-service asks auth-service for an agent's capabilities on first contact.

The distinction that matters: **an event is in the past tense and needs no reply**. If a service needs an answer before it can continue, that is a query and HTTP is honest about it.

28 What happens when a service goes down?

Depends which, and that asymmetry is the point:

- **assignment-engine down** — donations are still accepted. `donation.created` waits in Kafka and is consumed when it restarts. I test this by stopping the container, posting a donation (still 201), and starting it again.
- **Kafka down** — donations are still accepted, because the outbox pattern saves first and publishes second; a sweeper retries every 15 seconds.
- **Redis down** — matching genuinely stops. Redis is the system of record for live position, so `agent-location-service` reports **not ready**. This is the one hard dependency and I would not pretend otherwise.
- **auth-service down** — existing tokens keep working, because a signed JWT is self-contained. Nobody can log in.

29 Isn't Redis a single point of failure then?

Yes, for matching, and I would say so plainly. In production it would be Redis with replication and automatic failover — Sentinel or a managed cluster. What makes that acceptable rather than alarming is that **no durable data lives only in Redis**: positions are re-reported every 20 seconds, capabilities are a mirror of auth-service, and load counters can be rebuilt from the tracking records. Losing Redis costs availability, not data.

30 How do you trace a request across nine services?

A single `x-trace-id`, generated by the browser and **reused, never regenerated**, by every service it touches. So:

```
docker compose logs | grep <traceId>
```

follows one donation from the browser, through the gateway, into Mongo, onto Kafka, through the Go matching engine, into the status service. The `traceId` is shown on the monitoring detail view precisely so an operator can copy it.

That is the cheap version of distributed tracing. Jaeger or OpenTelemetry gives you spans, timings and a waterfall; this gives you `grep`. At nine services it is enough, and I would reach for OTEL when it stops being enough.

31 Reusing the trace id sounds minor. Why does it matter?

Because a service that generates a *new* id breaks the chain at exactly the point you care about. If donation-service invented its own, you could follow a request to its edge and no further — and the interesting failures are

always downstream. This is also why the outbox publishes under the **original request's** `traceId`, even minutes later.

32 How is configuration handled?

Environment variables, read once at startup through a config module that **fails fast** if something required is missing or malformed. A service that boots with a missing `JWT_SECRET` and only discovers it on the first login has turned a startup error into a production incident.

Secrets live in `.env`, which is gitignored — I verified that with `git check-ignore` rather than trusting it.

33 How would you scale this?

Per service, according to what actually strains:

- **agent-location-service** takes the highest write volume — every active agent every 20 seconds. It is stateless, so it scales horizontally behind the gateway.
- **assignment-engine** scales by **Kafka partitions**. Events are keyed by `donationId`, so all events for one donation land on one partition and are processed in order; adding partitions and instances adds parallelism without breaking per-donation ordering.
- The Node services are stateless and scale the same way.

The ceiling is Redis and Mongo, and I would hit read replicas before anything else.

34 Why key Kafka messages by donationId?

Because Kafka guarantees ordering **within a partition**, not across a topic. Keying by `donationId` puts every event for one donation on the same partition, so `donation.assigned` can never be processed after `donation.accepted` for the same donation. Without the key, two consumers could see one donation's lifecycle out of order and the status machine would reject legitimate transitions.

35 What's the versioning story for events?

Every event carries an `eventId` and a schema that only ever grows — **additive changes only**. A consumer ignores fields it does not know, so publishing a new field breaks nothing.

What I have *not* built is a schema registry, and I know what it would buy: compile-time enforcement that a producer cannot ship an incompatible change. Today that is enforced by discipline and code review, which does not scale past a small team.

A concrete case of this: `title` and `quantity` were added to tracking records in Phase 9. Older records lack them, and the UI handles their absence.

36 Could you backfill those old records?

Yes — reset the consumer group's offsets and replay from the beginning. The **idempotency guards make that safe**, which is the whole reason it is an option rather than a rewrite. I have not done it because the affected records are test data and I would rather the gap be visible than paper over it.

37 What's the deployment story?

`docker compose up -d --build` — 15 containers, one command. Multi-stage builds throughout; the Go services ship a binary rather than a toolchain, which is **30MB against 286MB**.

Health is one endpoint: `GET /ready` on the gateway fans out to all seven core services and returns 503 if any is down, so a script or orchestrator waits on one endpoint instead of polling seven.

38 Why is analytics-service optional?

It is behind a Docker Compose **profile** (`--profile analytics`), because Cassandra is heavy and the system works completely without it. That has a consequence the gateway handles explicitly: its route may have nothing behind it, so the gateway returns **502 with an explanation** rather than failing to start. "Not deployed" is a legitimate state, not an error.

D — Kafka and event-driven design (39–56)

39 Explain Kafka to someone who has never used it.

It is an **append-only log** that many readers can read independently.

A producer appends a message to a topic and is finished. Consumers each keep their own position — an **offset** — and read forward from it. Nothing is removed when it is read; it stays for the retention period.

That last property is what makes it different from a queue. With a traditional queue, one consumer takes a message and it is gone. With Kafka, three different services can read the same message for three different purposes, and a fourth can start next month and read the entire history from the beginning.

40 Why Kafka rather than RabbitMQ?

For **replay**, which is a property of the log, not of the messaging.

The convincing demonstration is analytics-service: it was written last, started with `fromBeginning`, and reconstructed **133 events across four days** — including days before it existed. Nobody wrote a migration. **The log was the migration.** With RabbitMQ those messages were consumed and gone.

If I only needed work distribution with per-message acknowledgement and complex routing, RabbitMQ would be the better fit and simpler to run.

41 Why Kafka rather than just HTTP calls between services?

Two reasons, one defensive and one that turned out to matter more:

1. **A donation is never lost because a downstream service is down.** With a direct call, donation-service fails when the engine is restarting and the donor is told to try again — for something the system could simply have accepted.
2. **Adding services became free.** tracking, notification and analytics were each added without changing a line of any existing service.

42 What is a consumer group?

A set of consumers that **share** a topic's partitions — each partition goes to exactly one member, so work is divided and each message is processed once by the group.

The important consequence: **separate groups each get the full stream.** `assignment-engine`, `tracking-service`, `notification-service` and `analytics-service` are four groups, so all four see every event. Adding a fifth takes nothing from the other four.

43 What delivery guarantee do you have?

At-least-once. Consumers do the work and *then* commit the offset, so a crash in between causes redelivery rather than loss.

The alternative ordering — commit first, then work — gives at-most-once and silently loses a donation on a crash. Between "sometimes twice" and "sometimes never", twice is recoverable and never is not.

Exactly-once would need Kafka transactions across the consume-process-produce cycle, plus a transactional sink. Since the work here ends in Mongo and Redis, idempotent handlers get the same practical result for far

less machinery.

44 So how do you stop a donation being assigned twice?

Every consumer deduplicates on `eventId` using Redis `SETNX`, namespaced per service so each of the four consumers gets its own turn at the event:

```
SETNX processed:<service>:<eventId> 1
```

If it returns 0, this event has already been handled and the consumer commits and moves on.

With one subtlety that caused a real bug: the marker is set *before* the work, so a **failure must release it**. Otherwise a crash mid-processing leaves the event marked as done and the retry skips a donation nobody ever handled.

45 Tell me about the dedup bug.

This is a good one to volunteer.

I originally read `eventId` from the **Kafka message header**. Every test passed. Then I replayed a topic to test recovery and the donation was **assigned twice** — because a replayed message **carries no headers**. The dedup key was empty, `SETNX` succeeded, and the event looked new.

Fixed by reading `eventId` from the **message body**, where it belongs: it is part of the event's identity, not transport metadata. The lesson is that a header is a property of *this delivery*; the body is a property of *the fact*.

46 What's the dual-write problem, and how did you solve it?

Creating a donation means writing to Mongo **and** publishing to Kafka, and no transaction spans both. Crash in between and you get a donation that nothing downstream ever hears about — the donor sees success and their food is never collected. Which is the worst possible failure, because it is silent.

Solved with a **transactional outbox**:

1. Save the donation with `eventPublished: false` — one atomic write.
2. Publish to Kafka.
3. On success, set `eventPublished: true`.
4. A sweeper every 15 seconds finds anything still false and republishes it.

Verified by stopping Kafka, posting a donation (still 201), restarting Kafka, and watching the sweeper publish it under the request's **original** `traceId`.

47 Why not two-phase commit?

Because it needs both participants to support a coordinator protocol, it holds locks across a network round trip, and it turns two independent availability problems into one joint one. The outbox gets the same guarantee — the event will eventually be published — using only a database write the service was making anyway.

48 Doesn't the outbox mean duplicates?

Yes, and that is the deliberate trade. If the publish succeeds but the flag update fails, the sweeper republishes. The outbox guarantees **at-least-once**, and the consumer-side idempotency absorbs the duplicate. The two halves are designed together — an outbox without idempotent consumers just moves the bug.

49 What was the outbox bug you found?

The first version treated a **skipped** publish as a success. Kafka was unreachable, the producer helpfully returned without throwing, and the code set `eventPublished: true`. So the donation was marked published when nothing had been published — and **the sweeper would never retry it**, because the sweeper's only signal is that flag.

One misclassified return value silently disabled the entire recovery mechanism it was meant to guarantee.

50 Walk me through the timeout mechanism.

There is no user request to hang a 90-second timer on, and an in-memory timer dies with the process. So deadlines live in a **Redis sorted set**:

- When an offer is made, `ZADD offers:deadlines <expiryTimestamp> <donationId>`.
- A watcher goroutine polls with `ZRANGEBYSCORE ... 0 <now>` for anything due.
- For each, it publishes `donation.timeout`.
- The engine consumes that, **adds the non-responders to an exclusion set**, and re-scores at the next radius on the ladder.

A sorted set is the right structure because the score is a timestamp, so "what is due now?" is a range query on a sorted index — not a scan.

51 Why exclude the agents who ignored it?

Because the top three are deterministic. Without an exclusion set, a timeout would re-score, find the **identical** top three, and offer it straight back to the same people who just ignored it — forever. The exclusion set is what makes the retry make progress instead of looping.

52 What if the watcher misses a deadline?

Nothing is lost, because the sorted set is **state, not a notification**. A deadline that came due while the watcher was restarting is still in the set with a score in the past, so the next poll picks it up. The cost of downtime is latency, not a stuck donation. That is the advantage over an in-process timer, which would simply have vanished.

53 What happens to an event you cannot parse?

It is logged and **committed** — deliberately, and I am not fully happy with it.

The alternative is not committing, which means Kafka redelivers the same unparseable message forever and **blocks the partition**, taking down every donation behind it. So the choice is lose one message or lose the topic.

The right answer is a **dead-letter topic**: move the message aside, commit, and keep a record for a human. That is the top item on my list.

54 Which events does each service consume?

Service	Consumes	Why
assignment-engine	<code>donation.created</code> , <code>donation.timeout</code>	To score and re-score
tracking-service	all six	It is the status authority
notification-service	<code>assigned</code> , <code>accepted</code> , <code>unassigned</code>	The ones a human should hear about
analytics-service	all six, <code>fromBeginning</code>	It is a historical record

55 Why does tracking-service own status rather than donation-service?

Because donation-service is the **write model** for a donation's content, and status is a **read model** built by folding events. donation-service sets `PENDING_ASSIGNMENT` at creation and then never hears about the rest.

Which means **donation-service's own status field goes stale**, and I left it there deliberately with a test asserting it is stale — so it is documented rather than forgotten, and so nobody wires the UI to it by accident. The UI reads tracking-service.

That is CQRS in a small way, and the honest version is that the stale field is a wart. Removing it would be cleaner; keeping it tested is the compromise.

56 How is the status machine implemented?

As a **pure function** in `statusMachine.js` that imports nothing. Given a current status and an event, it returns the next status or an illegal-transition error. Terminal states — `COLLECTED` , `CANCELLED` , `EXPIRED` — accept nothing.

Two things I got wrong and fixed:

- **Check ordering.** A duplicate `donation.accepted` reported `ILLEGAL_TRANSITION` , so every routine Kafka redelivery logged a warning. Now "already in this status" is checked **before** legality, which makes a redelivery a silent no-op — which is what it is.
- Terminal-state protection is tested against a **replayed** event, not just a wrong one.

E — Redis (57–72)

57 What is Redis doing in this system?

Four genuinely different jobs, which is the interesting part:

Use	Structure	Why Redis specifically
Find nearby agents	GEO set	Sub-millisecond radius query vs loading every agent and computing distance
Load counters	Integer	INCR / DECR are atomic — concurrent updates cannot be lost
Geocode cache	String + TTL	An address does not move; a 30-day TTL makes the cache pay for itself
Dedup + claim lock	SETNX	Answers "am I first?" atomically

The point worth making: it is not "we added Redis for caching". Only one of those four is a cache.

58 Walk me through the Redis keys for an agent.

agents:live	GEO set	every agent's position, one member each
agent:<id>:caps	HASH	capabilities, rating, contact
agent:<id>:load	counter	pending pickups
agent:<id>:alive	string+TTL	presence heartbeat, 120s

The fourth exists only to solve a constraint, which is the good question.

59 What constraint?

A Redis sorted set cannot expire individual members. A GEO set is a sorted set underneath. A whole *key* can have a TTL, but a *member* cannot.

So there is no way to say "forget this agent's location in two minutes". Left alone, an agent who closes the app stays on the map forever at their last position, and the engine cheerfully offers them donations they will never see — each one timing out and delaying the donation by exactly the minutes that matter most for hot food.

60 So how did you solve it?

Two parts:

1. **A separate per-agent key that does expire** — `agent:<id>:alive`, TTL 120s, refreshed on every location report. Every read checks it, so a stale agent is filtered out even while still in the geo set.
2. **A reaper goroutine** that periodically removes geo members whose heartbeat is gone, so the set does not grow without bound.

The check on read is what makes it *correct*; the reaper is what stops it leaking. Both are needed.

61 Why 20-second reporting against a 120-second TTL?

Six chances to survive a lapse — a tunnel, a lift, a few seconds of no signal. Reporting every 110 seconds would technically satisfy the TTL and would drop the agent out of matching the first time a single request failed.

The ratio is the point: the reporting interval should be a small fraction of the expiry, not just under it.

62 How does GEOSEARCH work?

Redis encodes each position as a **geohash** and stores it as the score in a sorted set. A geohash interleaves latitude and longitude bits so that nearby points share a long common prefix — which means a 2D proximity query becomes a set of 1D range scans over a sorted index.

`GEOSEARCH ... BYRADIUS 5 km ASC` returns members within the radius, nearest first, with distances computed for you.

63 What's the catch with geohashes?

Edge effects. Two points either side of a cell boundary can be physically close but share a short prefix, so a naive single-range scan would miss them. Redis handles it by searching the target cell plus its eight neighbours, which is why the results are correct rather than approximate — but it is worth knowing that "one range scan" is a simplification.

64 Why is agent-location's Redis a system of record rather than a cache?

Because there is nowhere else the position exists. A cache implies a slower source of truth to fall back to; there is none. That is why the service's `/ready` reports **not ready** when Redis is down, unlike geocoding-service, where Redis is genuinely just a cache and a miss simply costs an API call.

Same technology, two completely different criticalities. Being clear about which is which is what makes the readiness checks meaningful.

65 Describe the geocoding cache.

Cache-aside with `SETEX` and a 30-day TTL, keyed on a normalised address — lowercased and whitespace-collapsed, so `"12 MG Road"` and `"12 mg road "` are one entry.

Three refinements worth naming:

- **Negative caching.** A failed lookup is cached as `{ notFound: true }` with a shorter TTL — **1 hour** against the 30 days for a hit. Without it, a typo'd address that nobody can resolve hits the paid API on every retry. The shorter TTL is because a "not found" is far more likely to become wrong than a successful resolution.
- **Single-flight.** Concurrent requests for the same uncached address collapse into one upstream call instead of N.
- **The response says** `cached: true`, which the UI surfaces — partly for honesty, partly because it made the cache trivially testable.

66 Why 30 days rather than forever?

Because an address does not move but the *data about it* does — new buildings, renamed roads, corrected coordinates. Thirty days bounds how stale an answer can be at a negligible cost, since a repeated address is almost always repeated within days.

67 Why is the load counter a separate key rather than a field in the caps hash?

Because `INCR` and `DECR` are **atomic** and a read-modify-write on a hash field is not. Two services adjusting load concurrently with `HGET` then `HSET` would lose an update. `INCR` cannot.

68 What bug did the load counter have?

Kafka delivers at least once, so a **duplicate decrement** could push the counter negative. Which is worse than it sounds: load feeds the score as $1/(1+\text{pending})$, so a negative count makes an agent look **more** available the more events were replayed — exactly backwards.

`DecrementLoad` now clamps at zero and resets the key when it would go negative.

69 Explain the claim lock.

```
SETNX donation:<id>:claimed <agentId>
```

Atomic, so exactly one of three simultaneous accepts returns true. The losers get 409. The key also stores *who* won, so a retry from the winner is idempotent — they get success rather than a confusing conflict.

70 Why not a distributed lock like Redlock?

Because this is not mutual exclusion over time — it is a **one-time claim**. Redlock solves "hold a lock while I do work, and release it safely", with all the clock-skew and fencing problems that come with it. Here the winner is decided by a single atomic write and never needs releasing. `SETNX` is the whole solution and adding Redlock would be strictly worse.

71 All the offer keys — what are they?

```
processed:<service>:<eventId>    dedup marker
donation:<id>:offer               the offer record (who, ranks, breakdowns)
donation:<id>:claimed             the claim lock
donation:<id>:declined            the exclusion set for re-scoring
offers:deadlines                 sorted set of expiry timestamps
```

72 assignment-engine reads Redis keys that agent-location-service writes. Isn't that a boundary violation?

Yes, and it is the one deliberate exception in the system, so I would rather raise it than be caught by it.

Why: this is the hot path. An HTTP hop would add latency to a query Redis answers in under a millisecond, on the one path where latency costs food quality.

What it costs: the **key layout is now a contract between two services**. A rename in one silently breaks the other. It is documented in both, and asserted by both smoke suites, so the contract is at least tested.

If it became a problem, the clean fix is a shared client library owning the key schema — so the contract lives in code rather than in two comments.

F — The matching algorithm (73–92)

73 Explain the scoring formula.

```
score = 0.35 x distance      (1 - distanceKm/radiusKm)
        + 0.25 x categoryFit (compatibility matrix, floor 0.35)
        + 0.20 x load        (1/(1 + pendingPickups))
        + 0.15 x rating      (rating/5)
```

Every term is normalised to 0–1 **before** its weight is applied.

74 Why normalise first?

Because the terms are in incompatible units — distance in kilometres, load is a count, rating is out of five. Combined raw, whichever had the largest numeric range would dominate regardless of the weight written beside it. A 40km distance and a rating of 4 are not comparable numbers.

Normalising makes the weight mean what it says.

75 Justify each weight.

- **Distance 0.35** — it decides whether food arrives while it is still worth eating. The largest single factor, and still only about a third of the decision.
- **Category fit 0.25** — a close agent who cannot keep food at temperature delivers something nobody can serve. A failed delivery is worse than a slower one.
- **Load 0.20** — so donations spread across agents instead of stacking on whoever is nearest. Same reason a ride-hailing app does not hand you five rides at once.
- **Rating 0.15** — smallest on purpose. It is the least direct evidence about *this* pickup, and over-weighting it entrenches early winners and starves new agents of the work they need to build a record.

76 Are those weights tuned?

No, and I will not claim they are. They are hand-picked heuristics with stated reasoning. There was no accept/reject history to learn from when I wrote them.

What I did instead is make them learnable later: Phase 11 collects exactly that data — `acceptanceRate` per agent, time-to-accept, offer rounds — so a v2 could fit them with logistic regression or a learning-to-rank model. Saying "these are tuned" would be false.

77 The weights sum to 0.95, not 1.0. Is that a bug?

No, and it is a good catch. The project plan specified those four exact values, so I kept them rather than quietly rescaling to make the arithmetic tidy.

It **changes no ranking** — every candidate is scaled by the same constant, so the ordering is identical. It only matters when a *human* reads a score, which is why the UI always shows `0.9022 / 0.95` rather than a bare number. "0.90" on its own invites "out of what?", and 0.95 is the honest denominator.

78 The plan said `1/normalized_distance`. You didn't implement that. Why?

Because taken literally it is **unbounded**. An agent standing at the pickup point scores infinity and the other three terms stop mattering at all. It also contradicts the same document's instruction to normalise every term to 0–1.

I implemented a linear falloff, `1 - distance/radius`, which is what that instruction actually asks for: 1.0 at the pickup point, 0 at the edge of the search radius, bounded throughout.

This is documented in the code and in `ARCHITECTURE.md` as a deliberate departure, not a mistake.

79 Why is distance relative to the search radius rather than absolute?

Because the same distance means different things in different rounds. At the 5km rung, 4km away is nearly the edge and should score low. At the 50km rung, 4km is excellent. Scoring against the radius makes the term mean *"how good is this within the search we are actually doing"*, which is the useful question.

80 Explain the compatibility matrix.

Equipment	Cooked	Perishable	Bakery	Packaged
Insulated + refrigerated	1.00	1.00	0.70	0.55
Insulated only	1.00	0.75	0.75	0.60
Neither	0.40	0.45	1.00	1.00

It encodes two ideas: **capability** and **specialisation**.

81 Why does a better-equipped agent score LOWER on packaged goods?

This is the most interesting line in the table and usually the best follow-up.

An agent with an insulated box scores 0.55 on tinned goods; one with nothing scores 1.00. That looks backwards until you consider the alternative: specialised agents would win **everything**, and then the one hot meal that genuinely needs an insulated box would find them all busy carrying tins.

The matrix is not measuring who *can* carry the food. It is allocating a scarce resource to where it is uniquely needed.

82 Why is the floor 0.35 rather than 0?

Because a zero would make **every** candidate score zero in an area with no equipped agents, and the ranking would collapse into noise — arbitrary ordering among a field of zeros, at the exact moment you most need a sensible answer.

A floor keeps the other three terms meaningful when nobody is a good category match.

83 Why is urgency not a fifth scoring term?

Because it says nothing about which **agent** is better suited. It is a property of the donation, identical for every candidate — so as a score term it would shift all candidates equally and change no ranking, or worse, let an urgent donation outrank a much closer agent, which is backwards.

Instead urgency sets **time**:

Urgency	Categories	Window	Radius ladder
HIGH	cooked, perishable raw	90s	5 → 12 → 25 → 50 km
MEDIUM	bakery, beverages	3 min	5 → 10 → 20 → 35 km
LOW	packaged	5 min	5 → 8 → 12 → 20 km

84 Why do the ladders differ in shape, not just size?

Because the right search strategy depends on how much time you have.

HIGH jumps far, fast — for a cooling meal, an agent 25km away who says yes *now* beats a perfect match found after four patient rounds.

LOW widens gently — tins keep for months, so it is worth several careful rounds to find a genuinely well-suited nearby agent rather than sending someone across the city for baked beans.

85 What happens to an unknown category?

It is treated as **HIGH** urgency. If the guess is wrong the cost is a slightly hurried reassignment; the opposite mistake is letting food spoil while the system waits patiently. When you must guess, guess in the direction whose failure is cheaper.

86 Hard filters versus penalties — why filters?

A penalty only *lowers* a score; it does not remove anyone. So a penalised agent still wins when they are the only candidate — which is precisely the case where it matters most that they cannot do the job.

In the smoke test the **closest agent of all** carries only packaged goods. With a penalty they would still have won a hot meal.

Two things are hard filters: category eligibility and availability.

87 Why offer to three agents instead of one?

Because one agent might not answer, and asking them in sequence spends the food's remaining life on politeness. Three 90-second waits is four and a half minutes to reach the third person; asking all three at once costs 90 seconds total.

Three is a balance: enough that somebody probably answers, few enough that the notification is not spam and most agents who are asked have a real chance.

88 How are the three chosen, and what if scores tie?

Top three by score after filtering. Ties break deterministically — score, then nearer first, then by agent id:

```
if ranked[i].Score != ranked[j].Score { return ranked[i].Score > ranked[j].Score }
if ranked[i].Candidate.DistanceKm != ranked[j].Candidate.DistanceKm { ... }
return ranked[i].Candidate.AgentID < ranked[j].Candidate.AgentID
```

The id tie-break exists purely so ordering can never depend on map iteration or Redis reply order. **A flaky ranking is nearly impossible to debug from logs** — you cannot reproduce the decision you are trying to explain.

89 Why does the agent see "Ranked #1 for this pickup"?

Because it explains **why they were asked**, and it sets expectations. Being ranked #3 of 3 means two better-suited agents were asked first, so if the offer vanishes it is not a malfunction. Surfacing the ranking turns an opaque push notification into a comprehensible one.

90 `vehicleType` is collected but not scored. Why?

Because scoring it properly needs a **capacity model** the plan does not define — how many servings fit on a bicycle, at what point a van is required — and inventing those thresholds would be worse than the gap. A wrong capacity rule silently misroutes large donations.

It is captured so the data exists when there is a basis for the model.

91 How would you know if the scoring is any good?

The metric I actually collect is **first-round match rate** — how often the first batch of three says yes. It is the most direct evidence that the ranking picks the right agents: if the top three frequently all decline, the scoring is choosing badly, not the agents being unhelpful.

Alongside it: average time from offer to accept, average offer rounds, and the breakdown of failure reasons — which separates "nobody was online" from "everybody declined", and those imply completely different fixes.

Properly, this wants an A/B test between weight sets, which needs more traffic than a project has.

92 How would you introduce machine learning here?

Carefully, and not by replacing the whole thing.

The narrow, well-posed problem is: **given a donation and a candidate, predict the probability they accept**. That is binary classification on data I now collect, and it slots in as a replacement for the hand-picked weights while the hard filters, the radius ladder and the timeouts stay exactly as they are.

Two things I would insist on. First, **the explanation has to survive** — a model whose ranking cannot be explained gives up the property I consider the project's best feature, so I would use something inspectable, logistic regression or gradient-boosted trees with feature attributions, not a black box. Second, **acceptance is not the goal** — optimising it alone would learn to offer only easy pickups near the city centre and quietly abandon hard ones. The real objective is food collected in time, with work spread fairly.

G — Go (93–102)

93 Why Go for exactly two services?

Both are on the hot path, and for different reasons:

- **agent-location-service** takes the highest-frequency write in the system — every active agent, every 20 seconds, forever. Almost pure I/O with tiny payloads.
- **assignment-engine** scores candidates and notifies the top three in parallel, with a background watcher polling deadlines concurrently.

Go fits both: cheap concurrency for the fan-out and the watcher, and a compiled binary with predictable latency and no GC pauses of the kind a busy Node event loop can produce.

The other benefit is deployment: **30MB images against 286MB**, via a multi-stage build that ships the binary rather than the toolchain.

94 Isn't mixing languages a cost?

Yes — two toolchains, two dependency systems, two idioms, and a smaller pool of people who can maintain both. I would not do it for five services.

What makes it acceptable here is that the **contract is the network**: JSON over HTTP and Kafka. Neither Go service exposes a Go-specific interface, so the polyglot boundary is exactly the same boundary the architecture already has.

95 How do goroutines help concretely?

The top-3 notification fans out in parallel rather than sequentially, so three offers are written in roughly the time of the slowest one instead of the sum. And the deadline watcher runs as a goroutine inside the same process — no separate scheduler container, no cron.

96 How do you avoid leaking goroutines?

`context.Context` everywhere. The watcher takes a context, `select` s on `ctx.Done()` alongside its ticker, and returns cleanly on shutdown. Every outbound call takes a context with a timeout, so nothing blocks forever on a dead dependency.

97 What does your Go error handling look like?

Explicit returns, wrapped with `%w` so the chain survives:

```
if err != nil { return fmt.Errorf("invalid REDIS_URL: %w", err) }
```

I find this genuinely better than exceptions for a service like this, because every call site has to state what happens on failure. The dedup-release bug in section D existed *because* a failure path was not thought about — and Go's verbosity is what makes that path impossible to overlook a second time.

98 How is the scoring package structured?

As a **pure package that imports nothing but the standard library**. No Redis client, no Kafka, no HTTP. Inputs are plain structs, outputs are plain structs.

The payoff is direct: **85 tests run with no infrastructure at all** — I wrote the entire package and got its tests passing with Docker stopped. When one fails, the algorithm is wrong. It cannot be a slow broker.

99 How do you structure tests in Go?

Table-driven, which suits scoring especially well — the compatibility matrix is literally a table, so the test is a table over it:

```
tests := []struct{ name string; category string; caps Capabilities; want float64 }{ ... }
for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) { ... })
}
```

The store tests are different: they run against **real dockerized Redis**, not a mock. The whole point of that package is `GEOADD` / `GEOSEARCH` semantics and per-member expiry — a mock would only assert that I call the functions I wrote, not that Redis behaves the way the design assumes.

100 How does structured logging work?

`log/slog` emitting JSON, with the `traceId` bound to the context so every line carries it automatically.

One bug worth mentioning: I had **duplicate JSON keys** — `"round"` twice in one line, and `"topic"` twice elsewhere — because a field was bound to the logger *and* passed at the call site. It parses, but log processors may drop or reorder duplicates, so you cannot rely on what you see. Fixed by renaming the bound fields.

101 How do you configure the Go services?

A config package that reads environment variables at startup with typed helpers, and **fails fast**:

```
if c.OfferBatchSize, err = positiveInt("OFFER_BATCH_SIZE", 3); err != nil { ... }
```

So `OFFER_BATCH_SIZE=banana` refuses to boot rather than silently defaulting. Defaults are in code and visible, which also documents them.

102 Would you use Go for the whole system?

For a system dominated by this kind of work, probably yes. What Node earns its place with here is ecosystem fit for the CRUD-shaped services — Mongoose, Multer, express-rate-limit, http-proxy-middleware are all mature and would each be more work in Go for no benefit.

The rule I would state: **Go where latency and concurrency are the problem, Node where the problem is plumbing to a database and a third-party API.**

H — MongoDB and data modelling (103–112)

103 Why MongoDB?

Mostly continuity — the original was a MERN app, and the document model genuinely fits: a donation has nested quantity, nested location, a photo array, and optional fields that vary by category.

The properties I actually use: flexible schema while iterating, **2dsphere geospatial indexes**, and the natural fit of embedding data that is always read together.

Where I would prefer Postgres: anything with real relational integrity. Users and donations have a genuine foreign-key relationship and Mongo will not enforce it for me.

104 Which databases exist?

Separate per service: `hh_auth`, `hh_donations`, `hh_tracking`, `hh_notifications`, plus the Cassandra keyspace `hungerheal`.

One Mongo **instance**, separate **databases**, no cross-database queries. In production these would be separate clusters; one instance is a local convenience, not the design.

105 Mongo is on port 27018. Why?

27017 was taken by a local `mongod` on my machine. Worth mentioning only because it is the kind of thing that wastes an afternoon if the port is hardcoded — and it is why the tests read a configurable URI rather than assuming the default.

106 Tell me about the geospatial bug.

The best Mongo story in the project.

Donations arriving **without** coordinates — where geocoding failed — returned 500 with `Can't extract geo keys`. Mongoose was writing a GeoJSON subdocument with `type: 'Point'` and no `coordinates`, because the nested schema's default materialised the object. A 2dsphere index rejects a Point with no coordinates, and the failure surfaced on *every* query touching the collection, not just the insert.

Fixed with a subdocument schema using `default: undefined`, so the field is genuinely **absent** rather than present-and-invalid.

The general lesson: **absent and empty are not the same thing**, and an index will enforce that distinction whether or not your code respects it.

107 What indexes did you create?

- **Unique on** `users.email` — enforced at the database, not just in validation, because two concurrent signups can both pass an application-level check and only the database can settle the race.
- **2dsphere on the donation location** for geospatial queries.
- **Compound** (`donorId`, `createdAt: -1`) for the donor's list, which is always "mine, newest first" — the sort direction is in the index so no in-memory sort is needed.
- **eventPublished** on donations, for the outbox sweeper's only query.
- **Unique on** `donationId` in tracking — one status record per donation, by definition.

- `(recipientId, readAt, createdAt: -1)` on notifications — the inbox query, "my unread, newest first", served entirely from the index.
- **A unique partial index on** `(sourceEventId, recipientId)` in notifications, which is the interesting one.

108 Why is that partial index interesting?

Because it makes duplicate notifications **physically impossible** rather than merely unlikely — it is a second, independent line of defence for idempotency.

Redis `SETNX` dedup is the first line, but it is application logic and it can fail: Redis could be briefly unavailable, or a release-on-failure path could be wrong (which is exactly the bug I hit in section D). The unique index means that even if dedup lets a duplicate through, the **database** refuses the second insert. An agent seeing the same collection request twice would have them tapping an offer that is already theirs.

It is **partial** — `partialFilterExpression: { sourceEventId: { $type: 'string' } }` — because not every notification originates from an event. Without the filter, every event-less notification would have `sourceEventId: null` and the second one would collide with the first on the unique constraint. A partial index only constrains the rows the rule actually applies to.

The general principle: **for a correctness guarantee, prefer a constraint the database enforces over logic the application remembers to run.**

109 How is the timeline stored?

As an **array embedded on the tracking document**, appended to as events arrive. Embedded rather than a separate collection because it is only ever read with its donation, and never queried across donations.

The trade: unbounded array growth. Here the bound is natural — a donation goes through a handful of transitions and then reaches a terminal state. If timelines could grow indefinitely I would move them out, because MongoDB documents have a 16MB limit and large arrays make every update rewrite the document.

110 How are uploaded photos handled?

Multer writes to disk, the path is stored on the donation, and nginx serves `/uploads`. Not GridFS and not the database — files in a database make backups enormous and add nothing here.

In production this is object storage with presigned URLs, so the service never handles bytes at all.

111 What was the file-upload bug?

Multer writes the file to disk **before** validation runs. So a request that failed validation had already created a file, and the controller — which never executed — was where I had put the cleanup. Every rejected upload leaked a file.

Fixed by moving cleanup into the **error handler**, which is the one funnel every failure passes through. The lesson: cleanup belongs where failures converge, not on the happy path.

112 How do you handle validation?

Schema validation at the edge of each service, returning **field-level details**:

```
{ "error": { "code": "VALIDATION_ERROR",
             "details": [{ "field": "quantityAmount", "message": "must be positive" }] } }
```

The UI renders those per field, which is the difference between a form that tells you what to fix and one that says "invalid input". Every error also carries the `traceId`.

I — Cassandra and analytics (113–120)

113 Why Cassandra when you already have MongoDB?

Different question shape. MongoDB holds **current state** and answers "what is true now?" well. Analytics asks "what happened, in order, over months?" — an append-only, immutable, ever-growing time series.

Cassandra is built for exactly that: writes go to a commit log and an in-memory structure with no read-before-write, so ingestion stays fast as the dataset grows past what one machine holds.

If I am honest: at this project's data volume, Mongo would cope fine. Cassandra is there because the *workload shape* is genuinely different and I wanted to model it properly.

114 What tables exist, and why that many?

donation_events	every event , partitioned by donationId
events_by_day	the same events, partitioned by day
assignment_outcomes	one row per assignment decision
agent_totals	counter table per agent
daily_totals	counter table per day

115 donation_events and events_by_day hold the same data twice. Isn't that wrong?

In a relational database it would be a normalisation error. **In Cassandra it is the design.**

You must know your queries before designing your tables, because a query that does not match the partition key means scanning every partition on every node. `donation_events` answers "everything about donation X" — partitioned by donationId. It **cannot** answer "what happened today?" without a full scan. So the same events are written again, partitioned by day.

You duplicate data to buy query performance. That is the trade Cassandra asks you to make explicitly, and stating it plainly is the point of the answer.

116 Explain partition key versus clustering key.

The **partition key** decides which node stores the row — it is the unit of distribution, and every efficient query names it. The **clustering key** decides the sort order *within* a partition, so range scans and "most recent N" are free.

Here: partition by `donationId`, cluster by `(occurredAt, eventId)`. The `eventId` in the clustering key is a **tie-break** — two events in the same millisecond would otherwise collide and one would overwrite the other, because Cassandra treats a full primary key as an upsert.

117 What's a counter table for?

Aggregates that must not be recomputed. `agent_totals` and `daily_totals` use Cassandra **counter** columns, incremented as events arrive, so "how many donations has this agent collected?" is a single-row read instead of an aggregation over history.

The catch: **counters are not idempotent**. A replayed event double-counts. Which is precisely why the dedup guard runs before the counter update, and why counters live in their own tables — Cassandra does not allow counter and non-counter columns in one table.

118 Why no multi-partition BATCH?

Because a Cassandra `BATCH` across partitions is **not** a performance optimisation and **not** a transaction — it makes the coordinator responsible for writes to many nodes, which increases latency and creates coordinator pressure. Its only guarantee is atomicity, not isolation.

Writes here go individually and concurrently. Batches are used only where all statements share one partition, which is the single legitimate case.

119 Tell me about the backfill.

The most persuasive demonstration in the project. analytics-service was written **last**, in Phase 11. It started with `fromBeginning` and reconstructed **133 events across four days**, including days before the service existed.

Nobody wrote a migration script. The log *was* the migration. That is the concrete payoff of an event log over direct service calls, and it is worth leading with because it is verifiable in one command.

120 What's wrong with the Cassandra setup?

No TTL. History grows forever. A real deployment would set a TTL or archive cold partitions to object storage. I left it because the retention policy is a product decision I did not have a basis to make, and inventing one would be worse than naming the gap.

J — Gateway, auth and security (121–132)

121 What does the gateway do?

Four things: routing to seven services, **verifying the JWT once at the edge**, rate limiting, and aggregating readiness. The browser knows one origin instead of seven ports, so there is no CORS to configure and no service ports exposed to the client.

122 If the gateway verifies the token, why do services verify it again?

Defence in depth. The gateway forwards the decoded identity as `x-user-id` and `x-user-role` headers, so services *could* trust them — and they do not, because then any service would be unprotected the moment something reached it without going through the gateway. A misconfigured network policy, a debug port, a future internal caller.

It also keeps every service independently testable, which is why the smoke suites from Phases 1–11 still hit services directly.

123 Explain your JWT design.

Signed with HS256, expiring in **7 days**, containing `sub`, `role`, `email`, `name`, `phone`, plus an `issuer` of `hungerheal-auth` which is checked on verification.

The claims are chosen so no service ever calls auth-service to ask *"who is this?"*. A signed token **is** the proof. That is the difference from a session id, which needs a lookup on every request and makes auth-service a dependency of every operation in the system.

On algorithm pinning — worth being precise here, because it is a question that rewards precision. The gateway pins it explicitly:

```
jwt.verify(token, secret, { algorithms: ['HS256'] })
```

which matters because a verifier that trusts whatever the token's own header claims can be handed `alg: none` and will validate an unsigned token. **auth-service's own verify does not pass `algorithms`** — it checks the issuer only. That is not currently exploitable, because `jsonwebtoken` restricts itself to the HMAC family when the secret is a string, so `alg: none` is refused anyway. But it relies on a library default rather than stating the intent, and I would pin it in both places. If an interviewer finds that, agreeing quickly is the right answer.

124 What's the downside of stateless JWTs?

You cannot revoke them. A stolen token is valid until it expires, and a logout is only a client-side delete. My mitigation is short expiry, which is a trade not a solution — short expiry means more frequent re-authentication.

The real answer is refresh tokens with a server-side revocation list, which reintroduces exactly the state JWTs were chosen to avoid. That is the honest shape of the trade-off: statelessness and revocation are in tension and you pick which you need.

125 Why carry `name` and `phone` in the token?

Convenience with a real cost, and I would name both. It saves a lookup when displaying who someone is, but it means **a renamed user carries a stale name until their token expires**. For a phone number shown to

an agent en route, that is a genuine correctness issue.

If I were revisiting it I would keep `sub` and `role` — the things authorisation depends on — and fetch the rest. Identity in a token should be what you authorise on, not a cache of the user's profile.

126 How does rate limiting work?

Two policies at the gateway:

- **General: 300/minute.** Generous, because a donor posting five photos is a legitimate burst.
- **Auth: 20/minute with `skipSuccessfulRequests`** — only *failed* attempts count. A legitimate user logging in repeatedly is never locked out; someone guessing passwords is stopped after twenty tries.

That second flag is the part worth explaining: it makes the limit target the attack rather than the traffic.

127 Why is rate limiting at the gateway rather than in auth-service?

Because one policy at the edge covers **every** service, including ones not written yet. Phase 1 deliberately left it out of auth-service with a note saying it belonged here; Phase 12 is where that came due.

The limitation: in-memory counters, so it does not hold across multiple gateway instances. Multi-instance needs a shared store — Redis, which is already there.

128 Why no `express.json()` in the gateway?

Because parsing the body **consumes the request stream**, and the proxy would then forward an **empty body** downstream. POSTs arrive with nothing in them and the cause is entirely non-obvious — a classic gateway bug.

The gateway routes and authenticates; it never needs to read a payload, so the stream is left untouched for the proxy to pipe.

129 Tell me about the gateway bug you hit.

The first run of the gateway smoke suite failed **16 of 28 checks** with `no route for POST /signup`.

Cause: `app.use(prefix, ...)` **strips the prefix** before the middleware runs. So by the time the proxy saw it, `/api/auth/signup` was already `/signup`, and my `pathRewrite` doing `path.replace(/^\/api\/auth/, '/auth')` matched nothing.

The fix was to stop *substituting* the public prefix and instead **prepend** the service's own base path:

```
pathRewrite: (path) => `${route.basePath}${path}`
```

Which is both correct and simpler — each route declares where it lands rather than encoding a regex that has to agree with the mount point.

How it was found is the interesting part. Every service had been proven directly, and every path had been proven through Vite's dev proxy. Neither could catch a gateway that mounts routes differently. **A new integration layer needs its own integration test** — "it worked through the other proxy" is not evidence.

130 Why is route order load-bearing?

`/api/monitoring` and `/api/tracking` both live on tracking-service, and Express matches prefixes **in order**. Registered the other way round, `/api/tracking` would swallow every monitoring request. There is a smoke

check for exactly that, because it is the kind of thing that breaks silently during an unrelated refactor.

131 What other security measures are in place?

- **bcrypt** for passwords at **10 rounds** (configurable, min 4 max 15) — deliberately slow, which is the point.
- **A constant-time-ish login path.** When the email does not exist, the controller still runs `bcrypt.compare` against a precomputed dummy hash, so the same ~10 rounds are burned either way. Without it, "unknown email" returns noticeably faster than "wrong password" — which **leaks whether an address is registered** even though both responses are an identical 401. There is a smoke check asserting the two responses are indistinguishable.
- **Helmet** for security headers.
- Passwords never in responses; there is an explicit test asserting `passwordHash` does not leak.
- Role checks on every protected route, server-side. The UI hiding a button is not authorisation.
- The monitoring routes are **all GET** — there is no write endpoint to abuse, which is a stronger guarantee than a permission check on one.
- Secrets in a gitignored `.env`, verified with `git check-ignore`.

132 What are the security gaps?

Named honestly:

- **No HTTPS locally** — TLS belongs at the ingress, but it means nothing here is encrypted in transit today.
 - **No token revocation**, as above.
 - **No account logout** beyond rate limiting, so a distributed attack from many IPs is not stopped by it.
 - **No audit log.** ADMIN reads are not recorded, and for a role that can see every donation in the system they should be.
 - **Uploads are not virus-scanned** and are served from the same origin.
 - **No CSRF protection** — not currently exploitable because auth is a Bearer header rather than a cookie, but that is a property of the current design, not a defence.
-

K — Frontend (133–138)

133 What's the frontend stack?

React 19 with Vite, React Router, Tailwind CSS v4, Framer Motion for animation, Lucide for icons, Leaflet with react-leaflet for maps. Served in production by **nginx**, not the dev server.

134 Why nginx rather than the Vite dev server?

Because the dev server compiles on demand, ships source maps and is deliberately unoptimised. Three things in the nginx config matter:

- **index.html is never cached.** It references the hashed asset bundles, so a stale copy points at assets that no longer exist — the classic "white page after deploy".
- **Hashed assets are cached for a year**, safely, because their names change every build.
- **SPA fallback** `try_files $uri $uri/ /index.html`, so a deep link is handled by the router rather than 404ing.

`/api` is proxied to the gateway, so the browser sees one origin. In development Vite's proxy plays the same role, which is why **the frontend code is identical either way** — it always calls relative `/api/...` paths.

135 How does the agent's offer countdown work?

A `useCountdown` hook ticking once a second against the offer's `expiresAt`, with a progress bar proportional to the **full window** so it reads as time running out rather than a number getting smaller. Under 30 seconds it turns urgent — that is the last third of a 90-second window.

Two details I would point at:

- An expired card **stays visible** for a moment rather than vanishing mid-tap, which would leave the agent unsure what happened.
- The expiry is authoritative **server-side**. The countdown is a display of a deadline Redis owns, not the deadline itself — otherwise a clock-skewed device could accept a dead offer.

136 How does the UI know when something changed?

Polling — every 10 seconds on the donor dashboard, 5 seconds for agent offers, 15 for accepted jobs. Offers are only polled **while on shift**, because polling for an off-shift agent is pure waste and they cannot be offered anything anyway with a lapsed heartbeat.

This is the honest weak point of the frontend. WebSockets or SSE would be correct: lower latency and far less wasted traffic. Polling is what I built because it is simple and stateless and works through the proxy without extra infrastructure. For a 90-second decision window a 5-second poll is adequate, and I would not defend it beyond that.

137 What accessibility work did you do?

Labels genuinely associated with inputs via a shared `Field` component, so it is consistent across six forms rather than remembered each time. A visible `:focus-visible` ring everywhere. A skip-to-content link. `aria-live` on toasts so they are announced without stealing focus. Alt text stored **next to the image choice** in one

map, so an image cannot be added without describing it. `prefers-reduced-motion` switches off every animation — they are all decorative, so it costs nothing.

138 Any frontend issues you know about?

- **The JS bundle is 679kB (210kB gzipped)**, over Vite's warning threshold. Leaflet and Framer Motion dominate. Route-level code splitting would fix it.
 - **Polling instead of push**, as above.
 - Two small bugs I found while rebuilding the UI: image previews leaked an object URL per preview because nothing revoked them, and the location picker's search was a `<form>` nested inside the donation `<form>`, which is invalid HTML.
-

L — Testing, debugging and war stories (139–150)

139 How do you test a distributed system?

Two layers with different jobs, and keeping them separate is the point:

Unit tests — 246, no infrastructure. The scoring package and the status state machine import nothing but the standard library. A failure means the algorithm is wrong; it can never be a slow broker. I wrote the whole scoring package with Docker stopped.

Smoke tests — 333 checks, 13 suites, against the real stack. They read events back out of Kafka, assert on raw Redis structures, and wait out an actual 90-second timeout. These catch what unit tests structurally cannot: wiring, config, serialisation, and the behaviour of the real broker.

140 Why not mock Redis and Kafka?

Because in the places that matter, the **behaviour of the real thing is what I am testing**. The store tests exist to verify `GEOADD` / `GEOSEARCH` semantics and that sorted-set members cannot expire individually. A mock would only assert that I call the functions I wrote — it would have confirmed my incorrect mental model instead of correcting it.

Mocks are right where the dependency is incidental. They are wrong where the dependency's semantics *are* the design.

141 What's the worst bug you found?

The one that got past every test: **an agent who was invisible to matching, for good**.

Symptom: an agent on shift, 380m from a cooked-food donation, carrying all five categories, insulated *and* refrigerated — never offered it. The engine log said:

```
NO_ELIGIBLE_AGENTS radiusKm: 50 candidatesFound: 1
```

`candidatesFound: 1` is the clue. The agent **was** found geospatially, at every radius up to 50km, then ruled ineligible. Redis showed why — their capability hash held exactly one field, `available: 1`. No categories. And category eligibility is a hard filter, so that is exclusion from everything.

142 What was the cause?

Three individually correct pieces and an invariant none of them owned:

1. `SetAvailability` does `HSET`, and `HSET` **creates the hash if missing**.
2. `HasCapabilities` was `EXISTS` on that key.
3. The capability mirror ran only on a location report, only when `HasCapabilities` was false.

So if availability arrived first, the stub satisfied the very condition that would have triggered the mirror — **permanently**. The agent sat on the map, heartbeat healthy, eligible for nothing.

And availability *did* arrive first, because the frontend set `sharing = true` synchronously on tap while `watchPosition` was still acquiring a fix. The UI said "On shift" and offered the availability toggle before the first location report had anything to send.

143 How did you fix it?

Five parts, and the shape matters: the first two make ordering **irrelevant** rather than making one order correct.

1. **A sentinel field** — `HasCapabilities` now checks `updatedAt`, which only the full mirror writes. `EXISTS` answered "is there a hash?" when the question was "is there a *complete* mirror?".
2. **Every first-contact endpoint mirrors**. The logic moved into `ensureCapabilities`, called from the location *and* availability handlers, before the availability write.
3. **available is live state, not registration data**. Once 1 and 2 landed, a mirror could flip an agent who had just paused back to accepting work, since auth-service defaults it true.
4. **A degraded write must not look finished**. This one I caught re-reading my own diff: when auth-service is unreachable the service stores name and phone from the token, and my new sentinel would have stamped that stub as complete — **reintroducing the original bug through a different door**. A separate `SavePartialProfile` deliberately omits the sentinel so it stays retryable.
5. **The shift starts at the first GPS fix**, not at the tap, with an `acquiring` state in between so the UI never claims an agent is matchable while the server has heard nothing.

144 Why did 333 tests miss it?

Because every test set an agent up the same way — location first, because that is the natural way to write it:

```
place_agent() { signup; report_location; } # mirror runs, caps complete
```

Nothing exercised availability-before-location. The suite was not weak; it had **encoded the happy path's sequencing as though it were the only sequencing**.

The lesson I took: **where two endpoints write the same key, tests have to cover both orders**. A UI change — or a slow GPS — is enough to swap them.

145 How did you verify the fix?

Four regression tests, and the three covering the original bug were each **confirmed to fail against the old code before I kept them**.

That step is not optional, and I had learned it the hard way earlier (question 147). One of the three *did* initially pass against the old code — because my test agent ids did not match the suite's `agent:test-*` cleanup pattern, so stale keys from the previous run carried the categories in. **The near-miss was the same failure mode as the bug itself**: state left behind by one path silently satisfying another.

Then end to end on the live stack: availability-first through the gateway now produces a complete record, and a donation 380m away **matched on round 1**.

146 You had two broken records in production. How did you repair them?

By deleting the incomplete key and letting the next request rebuild it. Safe precisely because the caps hash is a **mirror, not a source** — auth-service owns the truth, Redis is derived. Deleting a derived record is a cache invalidation, not data loss.

That property was a deliberate Phase 4 decision, and it is what made the repair a one-liner instead of a migration. The detection was possible because a stub is now distinguishable from a complete record:

```
[ "$(redis-cli HEXISTS "$k" updatedAt)" = "0" ] && redis-cli DEL "$k"
```

147 You mentioned learning something the hard way. What?

A test that **reported success for behaviour it never exercised**, which is worse than no test.

An idempotency smoke check was supposed to republish an event and assert the donation was not assigned twice. It passed. It was also meaningless: a literal `\n` had broken the producer command, and the output was discarded by `>/dev/null 2>&1`. **Nothing was ever republished.** The test asserted that nothing bad happened after doing nothing at all.

Fixed by asserting the replay's own exit status *first*, before drawing any conclusion from it. Ever since, I check that a new test fails against the broken code before I trust it — which is exactly what caught the near-miss in question 145.

148 What other bugs are worth mentioning?

Bug	Why it mattered
Ungeocoded donations 500'd	Mongoose wrote a Point with no coordinates; 2dsphere rejected it on every query
Skipped Kafka publish reported success	The outbox would never have retried those donations
Failed uploads leaked files	Multer writes before validation, bypassing the controller
Dedup read the Kafka header	A replay carries no headers → assigned twice
An idempotency check passed vacuously	It tested nothing
Gateway stripped its own prefix	16/28 checks failed; the rewrite matched nothing
A duplicate JSON log key	Processors may drop or reorder duplicates
<code>applyEvent</code> check ordering	Every routine redelivery logged a false warning
Load counter could go negative	Made an agent look <i>more</i> available the more events replayed

149 What do those bugs have in common?

Most of them are **a correct component with an incorrect assumption about its neighbour**. The dedup logic was right, but assumed headers survive a replay. The outbox was right, but assumed a return value meant success. The gateway routing was right, but assumed Express left the path alone.

Which is exactly the failure mode microservices introduce: each piece is testable and correct in isolation, and the bug lives in the gap between two of them. It is the argument for integration tests that cross the seam, and the reason the smoke suites exist at all.

150 What did you learn from this project?

Three things I would actually carry forward:

1. **The cost of distribution is paid in failure modes, not in code.** Writing nine services was not hard. Duplicate delivery, partial failure, two writers on one key, and eventual consistency are the real bill, and none of them exist in a monolith.

2. **A test you have not seen fail is not evidence.** Two of my worst moments were tests that passed while proving nothing. Now I break the code deliberately and watch the test go red before I believe it.
3. **Being able to explain a decision is a feature, and it has to be designed in.** Storing the losing candidates' scores cost almost nothing at the time and is the single most valuable thing in the system — it is what made a "wrong" assignment diagnosable instead of arguable. It is also what let me refuse an override button with a straight face.

And one thing about honesty: the strongest answers in this whole document are the ones that name a limitation — the weights are not learned, the polling should be WebSockets, the stale status field is a wart, and there is no partner directory because there is no partner model. Interviewers trust a candidate who volunteers the boundary of what they built.

Appendix — questions to ask them

Prepared questions signal you evaluate the role as much as they evaluate you:

- How do you decide a service is worth splitting out here?
 - What does on-call look like, and who carries it?
 - When something breaks in production, what do you reach for first?
 - How much of the code was written with AI assistance, and how has that changed code review?
 - What is the thing in the codebase everyone agrees should be fixed and nobody has?
-

Appendix — the three demos to have ready

If there is a laptop, these land harder than any answer:

1. **Equipment beating proximity.** Two agents, the closer one without an insulated box, cooked food posted between them. The farther, equipped agent wins — `equipped(0.809) beat barenear(0.735)` . This is the thesis in one line.
2. **The score breakdown.** Open the monitoring view on any donation: four weighted terms for every candidate, winners and losers. Then point out there is no assign button, and say why.
3. **Nothing is lost when a service dies.** `docker compose stop assignment-engine` , post a donation (still 201), start it again, watch it get matched. The difference between an event log and an HTTP call.

`docs/DEMO.md` has the full 10-minute script.